

```
import cv2

import numpy as np

import torch

# Load your trained YOLOv5 model here

model = torch.hub.load('ultralytics/yolov5', 'custom', path='best.pt')

# Specify the path to the video file

video_path = 'cam3.avi'

# Create a video capture object

cap = cv2.VideoCapture(video_path)

# Check if the video file is opened successfully

if not cap.isOpened():

    print("Error opening video file.")

    exit()

# Get video properties

fps = cap.get(cv2.CAP_PROP_FPS)

frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))

frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

# Define the codec and create VideoWriter object to save the output

output_path = 'output.mp4'

fourcc = cv2.VideoWriter_fourcc(*'XVID')
```

```

out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Calculate the center area position

center_area_y = frame_height // 4 # Move the area even higher (adjust this value as
needed)

center_area_height = frame_height // 8

center_area_start = (0, center_area_y)

center_area_end = (frame_width, center_area_y + center_area_height)


# Initialize counters and dictionaries

bottles_passed = 0

bottle_centers = {} # To track the centers of the bottles on the line

bottle_state = {} # To track the state of each bottle (visible in center area or not)


while cap.isOpened():

    ret, frame = cap.read()

    if not ret:

        break


    # Perform object detection on the current frame

    results = model(frame)


    # Get detected objects and their information

    detections = results.pandas().xyxy[0]

    for _, detection in detections.iterrows():

        label = detection['name']

        confidence = detection['confidence']

```

```

x_min, y_min, x_max, y_max = detection[['xmin', 'ymin', 'xmax', 'ymax']].astype(int)

# Draw bounding box and label on the frame

cv2.rectangle(frame, (x_min, y_min), (x_max, y_max), (0, 255, 0), 2)

cv2.putText(frame, f'{label} {confidence:.2f}', (x_min, y_min - 10),

            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

obj_id = detection.name

center_x, center_y = (x_min + x_max) // 2, (y_min + y_max) // 2

bottle_centers[obj_id] = (center_x, center_y)

if center_area_start[0] < center_x < center_area_end[0] and center_area_start[1] <
center_y < center_area_end[1]:

    # If the bottle is on the line and hasn't been counted yet, count it

    if obj_id not in bottle_state or not bottle_state[obj_id]:

        if (center_x, center_y) in bottle_centers.values():

            bottle_state[obj_id] = True

            bottles_passed += 1

# Draw the center area rectangle on the frame

cv2.rectangle(frame, center_area_start, center_area_end, (0, 0, 255), 2)

# Check if the center of the object is within the center area

if center_area_start[0] < center_x < center_area_end[0] and center_area_start[1] <
center_y < center_area_end[1]:

    # If the bottle is on the line and hasn't been counted yet, count it

    if obj_id not in bottle_state or not bottle_state[obj_id]:

```

```

        if (center_x, center_y) in bottle_centers.values():

            bottle_state[obj_id] = True

            bottles_passed += 1

    else:

        bottle_state[obj_id] = False

for obj_id, center in bottle_centers.items():

    center_x, center_y = center

    if center_area_start[0] < center_x < center_area_end[0] and center_area_start[1] <
center_y < center_area_end[1]:

        cv2.circle(frame, center, 5, (0, 255, 0), -1) # Change color to green if centroid is in
the ROI

    else:

        cv2.circle(frame, center, 5, (0, 0, 255), -1)

# Display the count on the video

cv2.putText(frame, f'Bottles Passed: {bottles_passed}', (50, 50),

            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

# Write the annotated frame to the output video

out.write(frame)

# Display the annotated frame (optional, for visualization purposes)

cv2.imshow('Object Detection', frame)

# Exit if 'q' key is pressed

```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

```
# Release the video capture and writer objects
```

```
cap.release()
```

```
out.release()
```

```
cv2.destroyAllWindows()
```